

Bitácora de sesión — mammoviz-tda

Resumen de trabajo del 17 de julio de 2026

Sesión asistida con Claude Code

2026-07-17

Punto de partida

Al empezar la sesión, el repositorio `mammoviz-tda` (TFG sobre Análisis Topológico de Datos aplicado a la clasificación de mamografías CBIS-DDSM) tenía:

- Memoria LaTeX con 8 capítulos, de los cuales los capítulos 7 (Resultados) y 8 (Conclusiones) eran plantillas con marcadores de posición (“–”, “*Redactar a partir de los resultados obtenidos*”).
- `Pipeline` en `src/` (`data.py`, `preprocessing.py`, `topology.py`, `models.py`, `evaluate.py`) sin ningún test automatizado.
- Un notebook de demostración (`01_pipeline_demo.ipynb`) que funcionaba, pero caía a un descriptor de respaldo (curva de Betti-0) porque `giotto-tda` no tiene binarios para Python 3.14 (versión instalada en esta máquina).
- Sin datos reales de CBIS-DDSM descargados (solo el manifiesto de TCIA).
- Dos figuras de la memoria eran cuadros de texto de relleno (“*sustituir por la figura definitiva*”), no imágenes reales.

Trabajo realizado

1. Capítulos 7 y 8 de la memoria

Se sustituyeron los marcadores de posición por contenido real, basado en la ejecución efectiva del *pipeline* (no en resultados inventados): diseño experimental (E1–E4), configuración, estado explícito de los datos pendientes de descargar, y una tabla de cumplimiento de objetivos (O1–O6) honesta sobre qué está implementado frente a qué falta validar con datos reales.

2. Tests unitarios (pytest)

Se creó el directorio `tests/` con suite completa para `src/`: `test_data.py`, `test_preprocessing.py`, `test_models.py`, `test_evaluate.py` y tests de topología divididos en `test_topology.py` (genérico), `test_topology_gudhi.py` y `test_topology_giotto.py` (específicos de cada backend, con `pytest.importorskip`). Se detectó y corrigió un error de diseño propio: un `importorskip` a mitad de fichero salta el **módulo entero**, no solo lo que viene después — de ahí la separación en varios ficheros. Resultado final: **43 tests pasan, 2 se saltan** (los que requieren TensorFlow, no instalado en este entorno).

3. Backend de homología persistente portable (gudhi)

`giotto-tda` solo publica binarios hasta Python 3.12. En vez de instalar una versión antigua de Python en paralelo, se identificó que `gudhi` + `persim` sí tienen binarios para Python 3.14 y calculan la misma homología persistente cúbica. Se reescribió `src/topology.py` para elegir automáticamente el mejor backend disponible (`giotto-tda` → `gudhi` → curva de Betti-0), con

una función `backend()` para consultarlo, y una implementación propia de imagen de persistencia (núcleo gaussiano ponderado por persistencia) para cuando el backend activo es `gudhi`.

4. Notebook y resultados actualizados

Se actualizó `01_pipeline_demo.ipynb` para usar `backend()` y reportar correctamente qué motor topológico está activo, y se volvió a ejecutar de extremo a extremo. Al pasar del descriptor de respaldo (Betti-0, solo H_0) a la imagen de persistencia completa (H_0+H_1 , backend `gudhi`), los resultados sobre el conjunto sintético mejoraron notablemente:

	Antes (Betti-0, solo H_0)	Después (gudhi, H_0+H_1)
Sensibilidad (Random Forest)	0.60	0.80
AUC (SVM / Random Forest)	0.74 / 0.78	0.844 / 0.833
Dimensión del vector	24	288

Cuadro 1: Comparación de resultados sobre el conjunto sintético (120 imágenes), antes y después de disponer de un backend TDA real.

5. Figuras reales en la memoria

Se sustituyeron los dos últimos cuadros de texto de relleno:

- El diagrama del *pipeline* (Cap. 5) ahora es un diagrama TikZ real (código, no imagen externa), corregido tras detectar que se salía del margen a tamaño natural (`resizebox`).
- Tres figuras extraídas de la ejecución real del notebook (Cap. 7): ejemplos sintéticos benigno/maligno, matriz de confusión del mejor modelo, y diagramas de persistencia H_0+H_1 de un caso de cada clase.

6. Apéndice ampliado

Se rellenó el apéndice (antes vacío/genérico) con una tabla real de los campos del manifiesto de TCIA (verificada contra el `.xlsx` presente en el repositorio), la descripción de los campos esperados en los CSV de casos (aún por descargar), y dos fragmentos de código relevantes (detección de backend, imagen de persistencia manual). De paso se detectaron y corrigieron dos tablas con columnas sin ajuste de texto que desbordaban la página (bug preexistente en el capítulo 8, agravado por la tabla nueva del apéndice).

Verificación

Cada cambio se verificó activamente, no solo se asumió correcto:

- La memoria se recompiló (`latexmk + biber`) tras cada tanda de cambios, comprobando 0 errores y 0 referencias sin resolver.
- Las páginas con figuras y tablas nuevas se renderizaron a PNG (`pdftoppm`) y se inspeccionaron visualmente para detectar desbordamientos de márgen, no solo confiar en que “compila”.
- La suite de tests se ejecutó de principio a fin varias veces (43 passed, 2 skipped, estable).

Estado del repositorio al cierre

Nada de esto se ha commiteado (a petición explícita). Ficheros modificados o nuevos en el árbol de trabajo:

```
M .gitignore
M README.md
M notebooks/01_pipeline_demo.ipynb
M requirements.txt
M src/topology.py
M tfg/capitulos/05-metodologia.tex
M tfg/capitulos/06-implementacion.tex
M tfg/capitulos/07-resultados.tex
M tfg/capitulos/08-conclusiones.tex
M tfg/main.pdf
M tfg/main.tex
?? conftest.py
?? pytest.ini
?? tests/
?? tfg/figuras/diagramas_persistencia_sintetico.png
?? tfg/figuras/ejemplos_sinteticos.png
?? tfg/figuras/matriz_confusion_sintetico.png
```

Pendiente (fuera del alcance de esta sesión)

Todo lo pendiente depende de un mismo bloqueo externo: la descarga real de CBIS-DDSM (CSV de casos + imágenes DICOM/máscaras vía NBIA Data Retriever, del orden de decenas de GB), que requiere cuenta en TCIA y no es automatizable desde este entorno. Una vez descargados los datos:

1. Ejecutar `src/topology.py` (ya operativo con `gudhi`) sobre las imágenes reales y rellenar la Tabla 7.2.
2. Entrenar la CNN de referencia (`src/models.py:build_cnn`) — requiere además instalar TensorFlow.
3. Ejecutar el experimento de fusión TDA + CNN.
4. Generar las visualizaciones finales (Grad-CAM vs. diagramas de persistencia) sobre casos reales.